arXiv:2405.08644v1 [cs.CL] 14 May 2024

Thinking Tokens for Language Modeling

David Herel, Tomas Mikolov

Faculty of Electrical Engineering, Czech Technical University in Prague
Czech Institute of Informatics, Robotics and Cybernetics, Czech Technical University in Prague
hereldav@fel.cvut.cz

Abstract

How much is 56 times 37? Language models often make mistakes in these types of difficult calculations. This is usually explained by their inability to perform complex reasoning. Since language models rely on large training sets and great memorization capability, naturally they are not equipped to run complex calculations. However, one can argue that humans also cannot perform this calculation immediately and require a considerable amount of time to construct the solution. In order to enhance the generalization capability of language models, and as a parallel to human behavior, we propose to use special 'thinking tokens' which allow the model to perform much more calculations whenever a complex problem is encountered.

1 Introduction

Language models based on neural networks have gained a great deal of interest in recent years [15, 14]. Their impressive and coherent answers amazed people across many industries. However, it has soon been discovered that these language models have problems with complex tasks [5, 4].

Complex questions such as *'how much is 56 times 37'* which are computationally requiring, are problematic for the language model to process or even answer correctly. One can argue that humans also cannot perform this calculation right away and require a considerable amount of time to provide a solution.

Although these reasoning abilities could be improved by employing a large amount of supervision by providing labeled examples to the model [4], we aim for a much faster unsupervised approach.

To enhance the generalization capability of language models, we propose to use special 'thinking tokens' which allow the model to perform much more calculations whenever a complex problem is encountered. This could result in an improved generalization capability of language models, which could adapt to more complex tasks and even decide themselves what strategy is most beneficial for an encountered problem.

Figure 1: Illustration of 'thinking tokens' (marked as <T>) in a sentence which requires a complex calculation and the positive impact of this approach on perplexity of the model (lower is better).

2 Related work

Research regarding reasoning can be traced back to 1959 [16], and now continues to be a big part of theorem proving [18, 6, 8]. Presently, large language models are being used to learn reasoning from natural language [1, 11].

A similar problem has also been studied in [9], where a language model recomputes only a part of the recurrent hidden layer. Another work with a similar motivation explores the possibility of using a neural network that is capable of learning algorithms [10].

3 Thinking tokens for language models

Our approach is to introduce special 'thinking tokens' ($< T >$) after each word in a sentence whenever a complex problem is encountered. The core idea is that each 'thinking token' would buy more time for the model before an answer is expected, which would be used to run additional computations to better answer a complex problem that was presented. This concept has a great potential in recurrent neural networks [3, 7] due to their architecture, because it enables the RNN to perform multiple in-memory operations in a single step, meaning that extra calculations can be run in the hidden layer multiple times.

As a proof of concept, we have added N 'thinking tokens' ($< T >$) after each observed word in a dataset. Our vision is that this basic concept can be extended to a self-adjusting model, which will be able to decide itself if and how many 'thinking tokens' will be used for a specific problem, where N could also vary throughout the sentence. This would allow us to reduce the computational time, which would not increase N times. The visualization of our core idea, which we aim to validate in this paper, is presented in Figure 1.

4 Results

Experiments execution has successfully produced numerous examples where the usage of 'thinking tokens' leads to an improvement in the model's judgment. Preliminary results show that sentences that require non-trivial reasoning, have the biggest improvement in perplexity when 'thinking tokens' are used compared to the standard model. This is also observable on the sample sentences from Maths dataset in Table 1. A larger scope of examples across all the datasets is in Appendix A.1.

We can observe that introduction of 'thinking tokens' is also successful for sentences that include specific numbers or representative symbols of numerical values.

Dataset	**Sentence**	**Ppl. orig. ↓**	**Ppl. <T> ↓**
maths	What is the remainder when 8922293 is divided by 263 ? 18	16.8	13.1
maths	Convert -464 (base 9) to base 6 . -1434	24.3	19.8

Table 1: Examples of sentences where the introduction of 'thinking tokens' is beneficial to the model. First and second column refer to the name of the dataset and the specific sentence. Two rightmost columns refer to the perplexity without tokens (Ppl. orig.) and with 'thinking tokens' (Ppl. <T>). A downward arrow ↓ indicates that lower is better.

5 Discussion and Future work

Language models often make mistakes in complex problems like calculations or reasoning, since they rely on large training sets and their great memorization capability. We show that giving RNNLM more time to 'think' and not pressuring the model to produce an answer immediately, helps the model resolve various complex tasks more accurately.

Building on the proof of concept, we plan to extend our research and create a model that would be able to decide itself how much extra time is needed in order to produce the best answer possible. If successful, this concept could be implemented as a default behavior for language models that encounter complex and computationally demanding tasks. We also believe that the ability of a model to self-regulate this factor would vastly improve adaptability and generalization capability of language models in general.

References

[1] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022.

[2] Russell Cooper and Andrew John. *Macroeconomics: Theory through applications*. Saylor Foundation, 2012.

[3] J Elman. Finding structure in time. *Cogn. Sci.*, 14(2):179–211, June 1990.

[4] Simon Frieder, Luca Pinchetti, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Christian Petersen, Alexis Chevalier, and Julius Berner. Mathematical capabilities of chatgpt, 2023.

[5] Timothy Gowers. It's amusing when ChatGPT makes ridiculous mathematical mistakes. But of course, it's more interesting to find out what it can do well. Here's one example that wasn't bad: I gave it a very rough outline of a proof and asked it to fill in the details. https://twitter.com/wtgowers/status/1611750773607604224, 2023. [Online; accessed 2023-02-27].

[6] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughin, Tat Thang Nguyen, and et al. A formal proof of the kepler conjecture. *Forum of Mathematics, Pi*, 5:e2, 2017.

[7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, nov 1997.

[8] Geoffrey Irving, Christian Szegedy, Alexander A Alemi, Niklas Een, Francois Chollet, and Josef Urban. Deepmath - deep sequence models for premise selection. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.

[9] Yacine Jernite, Edouard Grave, Armand Joulin, and Tomas Mikolov. Variable computation in recurrent neural networks, 2016.

[10] Armand Joulin and Tomas Mikolov. Inferring algorithmic patterns with stack-augmented recurrent nets. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 1*, NIPS'15, page 190–198, Cambridge, MA, USA, 2015. MIT Press.

[11] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models, 2022.